Memory Hierarchy · Cache · Virtual Memory

이후 진도 미리보기 (Lec1 교수 슬라이드의 topic 순서: memory hierarchy/cache → linking → ECF → VM → malloc) · **중간고사(Lec2~8) 범위 밖** · 교수 원본 슬라이드 공개 전이라 CS:APP 3판 / CMU 15-213 기준으로 구성, 강의 진도 나가면 원본 기준으로 보강

1 메모리 계층 (Memory Hierarchy)

- **핵심 직관**: 위로 갈수록 빠르고 · 작고 · (byte당) 비쌈. Lec4 슬라이드 그림의 계층(Register → Cache → Main memory → SSD/Disk)을 CS:APP 기준으로 확장한 것
- register는 사실상 1 cycle 접근. cache miss로 main memory까지 내려가면 수백 cycle (Lec4 슬라이드 기준)
- 아래 위젯에서 각 층을 클릭해 대표 크기·지연과 "L1 = 1초" 환산 스케일을 확인할 것

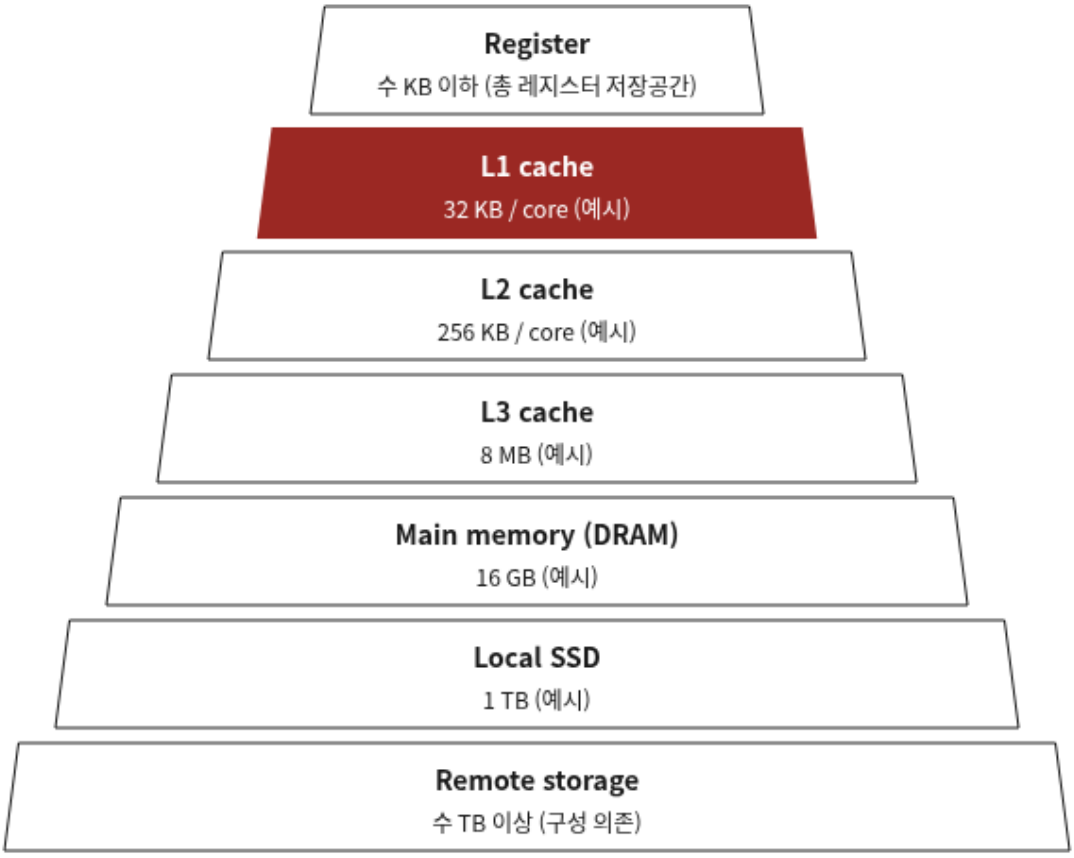
메모리 계층 탐색기

스케일 체험: OFF | 블록 이동 설명: OFF | 보기 초기화

위쪽 계층일수록 일반적으로 빠르고 작으며 bit당 비용이 높습니다. 층을 클릭하면 세부 정보가 표시됩니다. 크기와 지연은 대표적인 교육용 예시입니다.

Memory hierarchy

빠름 · 작음 · bit당 비용 높음



느림 · 큼 · bit당 비용 낮음

선택: L1 cache

크기 32 KB / core (예시) · 접근 지연 4 cycles

층을 클릭하여 선택

계층 선택

Register | L1 cache | L2 cache | L3 cache | Main memory (DRAM) | Local SSD | Remote storage

2. L1 cache

대표 크기	대표 접근 지연	지연시간 환산
32 KB / core (예시)	**4 cycles**	**1.333 ns**

CPU 코어 가까이에 있는 작은 cache입니다. 최근 사용하거나 인접한 데이터에 빠르게 접근하도록 돕습니다.

대표적인 L1 data cache hit 지연을 약 4 cycles로 가정합니다.

바로 아래의 L2 cache 접근은 L1 cache 접근보다 약 2.5배 지연됩니다 (4 → 10 cycles).

한 단계 위의 Register 접근과 비교하면 4배의 지연 배수입니다.

기준 CPU clock: 3 GHz. 접근 지연은 예시값이며, 계층마다 용량·지연·공유 방식이 다를 수 있습니다.

2 캐시 주소 해석 (Address → Tag / Set / Offset)

- 물리 주소는 **tag (t_cache bits) · set index (s bits) · block offset (b bits)**으로 쪼개진다. S = 2^s개 set, block 크기 B = 2^b
- 주소가 어느 set으로 가는지는 **set index bits**만이 결정. 같은 set 안에서는 tag 비교로 hit/miss가 갈림
- 주소 길이가 m bits이고 set index가 s bits, block offset이 b bits이면 tag는 t_cache = m − s − b bits이다. 따라서 주소는 tag, set index, block offset 순으로 분해된다.
- Set 수는 S = 2^s, block 크기는 B = 2^b bytes, set당 line 수는 E이다. 캐시의 데이터 용량은 C = S × E × B bytes이며, 이 계산에서는 tag·valid bit 등의 메타데이터를 제외한다.
- 바이트 주소 A에서 block offset은 A mod 2^b, set index는 $\lfloor A / 2^b \rfloor$ mod 2^s, tag는 $\lfloor A / 2^{s+b} \rfloor$로 계산한다.
- 직접 사상(direct-mapped, E=1)은 충돌이 잦고, E-way set associative는 그 충돌을 E개 라인으로 흡수

캐시 주소 해석기

주소 길이 m (3–64 bits): 32

set index 비트 (s): 4 → S=16

block offset 비트 (b): 4 → B=16

lines per set (E): 4

메모리 주소 (16진수): 0x1A3F

랜덤 주소

현재 설정: S=16 sets, B=16 bytes, E=4 lines/set · 총 데이터 용량 1024 bytes (tag/valid bits 제외)

tag | set index | block offset

tag	set index	block offset
0x1A	0x3	0xF
26 (10진) · 24 bits	3 (10진) · 4 bits	15 (10진) · 4 bits

메모리 주소 비트 분해

0x00001A3F

32-bit address · t=24, s=4, b=4

상위 비트 (MSB) 하위 비트 (LSB)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1	0	0	0	1	1	1	1	1	1

tag (24 bits) | set index (4 bits) | block offset (4 bits)

주소 분해 결과

tag (24 bits)	set index (4 bits)	block offset (4 bits)
0x1A	**0x3**	**0xF**
10진: 26	10진: 3	10진: 15

캐시 구조와 주소 매핑

set	line 0	line 1	line 2	line 3
set 0	cache line	cache line	cache line	cache line
set 1	cache line	cache line	cache line	cache line
set 2	cache line	cache line	cache line	cache line
set 3	**tag 비교 후보**	**tag 비교 후보**	**tag 비교 후보**	**tag 비교 후보**
set 4	cache line	cache line	cache line	cache line
set 5	cache line	cache line	cache line	cache line
set 6	cache line	cache line	cache line	cache line
set 7	cache line	cache line	cache line	cache line
set 8	cache line	cache line	cache line	cache line
⋮	⋮ 중간 set 생략			

E=4: set 3의 4개 line을 후보로 tag를 비교합니다.

set index는 후보 set을 정하며, hit 여부는 valid bit와 tag 비교로 결정됩니다.

주소 해석 공식

```
block offset = address mod 2^4

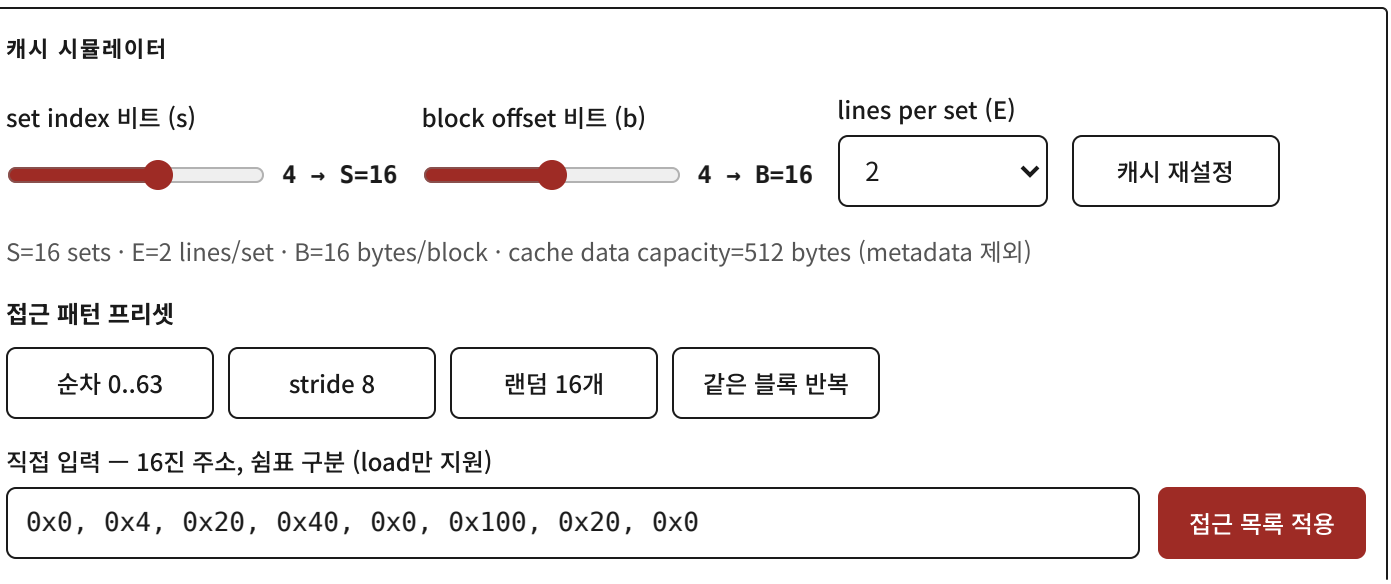
set index = floor(address / 2^4) mod 2^4

tag = floor(address / 2^8)
```

> **주소 0x1A3F에서 하위 4비트는 block offset 15이며, 그 위 4비트가 set index 3입니다. 따라서 set 3으로 매핑됩니다.**
>
> 계산: floor(6719 / 16) mod 16 = 419 mod 16 = 3.
>
> 4-way set associative이므로 해당 set의 4개 line에서 valid bit와 tag를 비교합니다.
>
> 해당 메모리 block의 시작 주소: 0x1A30 (block 크기 16 bytes).

3 캐시 시뮬레이터 (Hit / Miss / Eviction)

- 접근 순서(주소 trace)가 hit rate을 결정한다. Cache Lab (Part A/B) 전에 손으로 trace를 따라가 보는 연습용
- miss를 구분해 볼 것 — **Cache miss의 종류**
  - Cold (compulsory) miss: 해당 메모리 block에 처음 접근할 때 발생한다.
  - Conflict miss: 여러 block이 같은 set에 매핑되어 set의 E개 line을 초과하면서 발생한다. 같은 용량과 block 크기를 가진 fully associative cache에서는 hit였을 miss이다.
  - Capacity miss: 필요한 working set을 캐시 전체에 담을 수 없어 발생하며, 같은 용량의 fully associative cache에서도 발생할 miss이다.
  - Eviction: 해당 set의 모든 line이 유효한 상태에서 새로운 block을 적재하기 위해 기존 line을 교체하는 것이다.
  - Valid bit의 한계: Valid bit는 현재 line의 유효성만 알려 준다. 과거에 접근했지만 이미 캐시에서 쫓겨난 block인지까지는 알려 주지 않는다. 따라서 miss 종류는 전체 접근 trace를 분석하여 판별해야 한다.
- LRU: set이 꽉 차면 가장 오래 전에 쓰인 라인이 쫓겨남



시뮬레이션 실행

한 단계 | 끝까지 실행 | 처음으로

0 / 8 접근

hits	misses	evictions	hit rate
0	0	0	0.0%

현재 접근 주소

아직 실행한 접근이 없습니다. '한 단계'를 눌러 시작하세요.

`next: 0x0`

캐시 상태 (set × line)

set	line 0	line 1
set 0	V=0 · invalid	V=0 · invalid
set 1	V=0 · invalid	V=0 · invalid
set 2	V=0 · invalid	V=0 · invalid
set 3	V=0 · invalid	V=0 · invalid
set 4	V=0 · invalid	V=0 · invalid
set 5	V=0 · invalid	V=0 · invalid
set 6	V=0 · invalid	V=0 · invalid
set 7	V=0 · invalid	V=0 · invalid
set 8	V=0 · invalid	V=0 · invalid
set 9	V=0 · invalid	V=0 · invalid
set 10	V=0 · invalid	V=0 · invalid
set 11	V=0 · invalid	V=0 · invalid
set 12	V=0 · invalid	V=0 · invalid
set 13	V=0 · invalid	V=0 · invalid
set 14	V=0 · invalid	V=0 · invalid

set 15 V=0 · invalid V=0 · invalid

총 16개 set · 각 set 2개 line · t는 마지막 접근 시각

접근 결과 상세

첫 번째 메모리 접근을 실행하면 hit / miss 결과가 표시됩니다.

초기 상태: 모든 cache line의 valid bit = 0

> **캐시가 초기화되었습니다. 모든 line은 invalid 상태입니다.**
> 첫 번째 접근은 해당 block이 미리 적재되어 있지 않으므로 miss가 발생합니다.

접근 히스토리

HIT MISS MISS + EVICT 미실행

#1	#2	#3	#4	#5	#6	#7	#8
–	–	–	–	–	–	–	–
0x0	0x4	0x20	0x40	0x0	0x100	0x20	0x0

LRU: 각 set에서 마지막 접근 시각이 가장 오래된 valid line을 교체합니다. hit 시에도 마지막 접근 시각을 갱신합니다. 캐시는 처음에 모두 invalid 상태이며, write/modify 명령은 다루지 않습니다.

4 가상 메모리 (VA → PA Translation)

- 가상 주소는 **VPN (virtual page number) + VPO (page offset)**. 페이지 크기 P = 2^p면 VPO는 하위 p bits
- CPU는 먼저 **TLB**에서 VPN을 찾고, miss면 page table에서 PTE를 읽는다. PTE의 valid bit가 0이면 **page fault** (교육용 resident-bit 단순화: 실제로는 invalid PTE가 미매핑을 뜻할 수도 있고 fault가 항상 disk I/O는 아님). 시뮬레이터의 TLB는 cache-sim과 달리 **단순 교체 모델**(고정 way)이라 LRU가 아님에 주의
- 물리 주소는 PPN + VPO를 그대로 이어붙인 것 (PPO = VPO)

가상 주소 변환 시뮬레이터

VA 비트 수 m: 16

PA 비트 수 n: 12

page offset bits (p): 6 → P=64

TLB set index bits (s_TLB): 2 → T=4

TLB ways (E): 4

VA=16 bits · PA=12 bits · P=64 bytes · VPN=10 bits · PPN=6 bits · TLB=4 sets × 4 ways

시나리오 선택

TLB hit | TLB miss → page hit | page fault | 랜덤

가상 주소 VA (16진수)

0x08D5

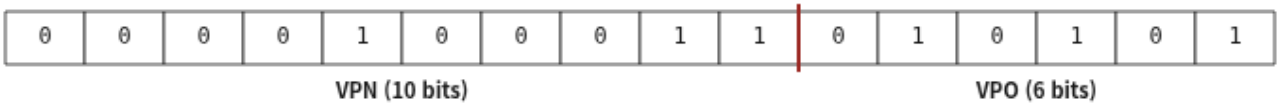
주소 적용

다음 단계 | 처음으로

① VA 분해 | ② TLB 조회 | ③ PTE 조회 | ④ PA 조립

① Virtual Address (VA)

16-bit VA **0x08D5**



다음 단계를 누르면 VPN과 VPO가 분리됩니다.

② Translation Lookaside Buffer (TLB)

set	way	V	TLB tag	PPN
3	0	1	0x8	0x3E
3	1	1	0x27	0x18
3	2	0	–	–
3	3	1	0x31	0x20
0	0	1	0x1	0x39
0	1	1	0x6	0x3D
0	2	0	–	–
0	3	1	0x10	0x5
1	0	1	0xC	0x2
1	1	0	–	–
1	2	1	0x16	0xA
1	3	1	0x1B	0xE

나머지 4개 TLB line 생략

TLB 조회 대기

③ Page Table (일부 PTE)

VPN	V	PPN
0x21	0	–
0x22	0	–
0x23	1	0x3E
0x24	1	0x3F
0x25	1	0x6
0x26	0	–
0x27	1	0x14
0x28	1	0x1B

page table 조회 전

④ Physical Address (PA)

주소 변환이 완료되면 PPN | PPO 비트열이 표시됩니다.

> **변환 시작 전**
>
> 가상 주소 0x8D5가 준비되었습니다. 다음 단계를 누르면 VPN과 VPO로 분해합니다.

교육용 단일 레벨 page table 모델입니다. TLB hit는 주소 변환 결과가 캐시되어 있다는 뜻이며, 실제 메모리 데이터의 cache hit를 의미하지 않습니다. Page fault가 발생하면 OS가 페이지를 메모리에 적재하고 page table을 갱신한 뒤 접근을 재시도해야 합니다. 이 과정의 실제 disk I/O와 OS 처리는 생략합니다.